UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI



Application Of Tile Transposition Ciphers In Anti-Scraping Image Protection System

Full Name	Student ID	Major
Nguyen Tung Duong	23BA14076	ICT
Tran Mai Anh	2410093	ICT
Pham Dinh Anh Duong	23BA14075	ICT
Ta Quang Huy	23BA14137	ICT
Ha Dang Huy	23BA14138	ICT
Nguyen Minh Duc	23BA14056	ICT
Nguyen Minh Luong	23BA14184	ICT

Academic Year: 2026 – 2027

Contents

1 Introduction **2**
- 1.1 Problem Statement: Automated Image Scraping and Economic Threats . . . 2
- 1.2 Project Objectives . . . 2

2 Theoretical Background **2**
- 2.1 From SRT to GRT . . . 2
- 2.2 Image decryption via Inverse GRT (IGRT) . . . 3
- 2.3 Key Terminology . . . 3

3 System Architecture & Application **4**
- 3.1 Secret Delivery . . . 4
- 3.2 Encryption & Decryption Lifecycle (Figure 3) . . . 4

4 Experimental Benchmarking & Analysis **4**
- 4.1 Decryption Recovery Analysis . . . 4
- 4.2 Computational Latency & Memory Overhead . . . 5
- 4.3 Ciphertext Compression Overhead . . . 6

5 Brute-Force Analysis & Limitation **6**

6 Conclusion **6**

1 Introduction

1.1 Problem Statement: Automated Image Scraping and Economic Threats

Images are widely stored and distributed via websites and Content Delivery Networks (CDNs), which enable images to be loaded and collected automatically at low processing cost [1]. This may allow unauthorized and illegal redistribution of images [2]. In addition, existing image-protection methods may involve a trade-off between protection and performance. In pixel-chaotic ciphers, shuffling individual pixels across the entire image destroys spatial correlation, leading to an increase in compressed file size. Therefore, websites need to balance image protection, bandwidth costs, and display performance.

1.2 Project Objectives

- Adapt and implement GRT (Generalized Rectangular Transform)-based tile-level image scrambling on the server and client-side viewport reconstruction (Section 3).
- Validate the reversibility of GRT and confirm pixel-perfect image recovery with no quality loss (Section 2.1).
- Measure computational latency, memory usage, and compression overhead across standard web formats (PNG, WebP, JPEG) to assess its feasibility for web-based image protection (Section 4).

2 Theoretical Background

2.1 From SRT to GRT

Special Rectangular Transform (SRT) is a transformation method used to rearrange pixel positions, allowing accurate image reconstruction through its corresponding inverse transformation (ISRT) [3]. The SRT matrix is expressed as:

$$R_{\text{SRT}} = \begin{bmatrix} 1 & r \\ s & rs+1 \end{bmatrix} \tag{1}$$

where r and s are two integer parameters that determine the transformation matrix. However, its transformation matrix is constrained because the upper-left element is fixed at 1, resulting in only two degrees of freedom. To address this limitation, we adopt the GRT to enhance the flexibility of the transformation matrix and introduces a translation offset (r_m, r_n) to displace the origin (Figure 2c).

Unlike SRT, GRT involves 4 parameters (a, b, c, d), providing 4 degrees of freedom and greater flexibility in the transformation matrix [3]:

$$\begin{bmatrix} x \\ y \end{bmatrix}_{n+1} = \left\{ \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}_n + \begin{bmatrix} r_m \\ r_n \end{bmatrix} \right\} \bmod \begin{bmatrix} M \\ N \end{bmatrix} \tag{2}$$

where (a, b, c, d) are 4 transformation parameters, $G = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ is a general 2×2 integer matrix whose determinant $\|G\|$ does not need to equal 1, $(x, y)_n$ are original pixel coordinates, $(x, y)_{n+1}$ are transformed tile coordinates, (r_m, r_n) are coordinate shifting offsets (Figure 2), and M, N are image height and width. However, the parameter set (a, b, c, d) must satisfy these 4 mathematical conditions [3].

1. **Condition 1** — $(b \bmod h = 0)$ **or** $(c \bmod v = 0)$**.**
2. **Condition 2** — $\gcd(\|G\|, p) = 1$**.**
3. **Condition 3** — $\gcd(a, h) = 1$**.**
4. **Condition 4** — $\gcd(d, v) = 1$**.**

where $p = \gcd(M, N)$ is the greatest common divisor of the grid dimensions, and $h = M/p, v = N/p$ represent the irreducible aspect ratio. These conditions ensure a one-to-one mapping across rectangular grids without tile collisions.

2.2 Image decryption via Inverse GRT (IGRT)

To restore the original coordinates $(x, y)_n$ from a scrambled tile $(x, y)_{n+1}$, we apply the 4-stage Inverse GRT (IGRT) from the corollary [3] (Figure 1):

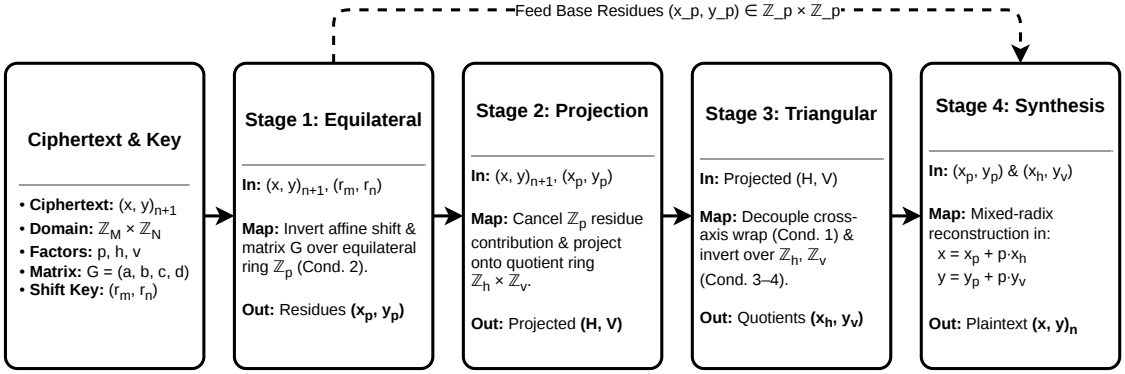
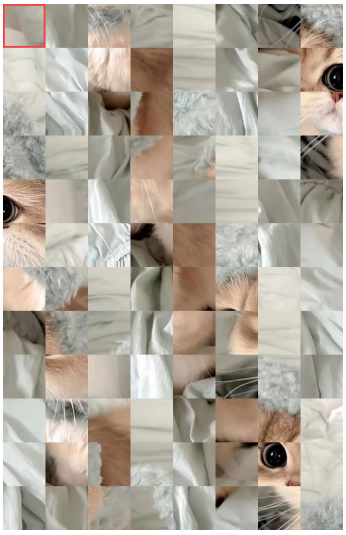


Figure 1: 4-Stage Single-Pass Coordinate Decryption in O(1) Time Complexity.

2.3 Key Terminology

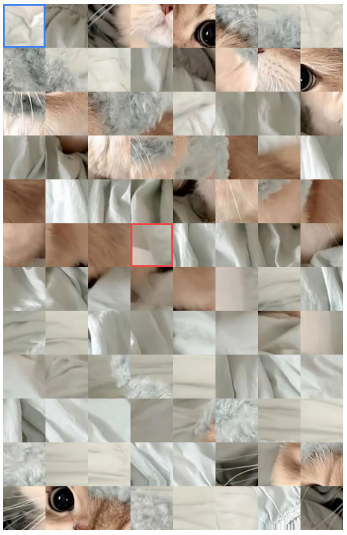
- **Pixel Permutation** rearranges pixel positions without changing their values. In GRT, a flexible transformation matrix is used to disrupt spatial correlations between neighboring pixels [3].
- **Tile-Based Scrambling** splits the image into several tiles and then permutes their positions. The original image can be reconstructed by restoring the tiles to their original positions.
- **Reversibility** allows the reconstruction of the original image, while **Bijective Mapping** ensures "one-to-one" correspondence between original and transformed positions.



(a) SRT: Tile $(0, 0)$ (red box) remains frozen at the top-left corner.



(b) Original unencrypted source image.



(c) GRT: $(0, 0)$ is displaced to (red box); a new tile (blue box) occupies $(0, 0)$.

Figure 2: Visual comparison of origin displacement between (a) Special Rectangular Transform (SRT), (b) the original image, and (c) Generalized Rectangular Transform (GRT).

3 System Architecture & Application

3.1 Secret Delivery

To protect web images, we propose breaking the pipeline as a symmetric permutation cryptosystem $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathrm{Enc}_K, \mathrm{Dec}_K)$ operating over a two-channel delivery protocol (Figure 3).

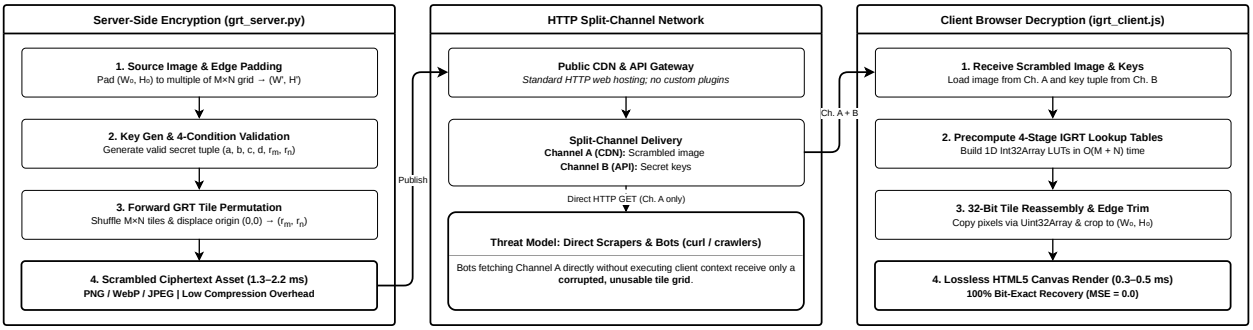


Figure 3: Cryptographic delivery across server, network, and client.

- **Channel A (Public CDN Asset):** Where automated bots, `curl` scripts, or crawlers fetching the image URL directly would receive only an unusable grid of shuffled tiles.
- **Channel B (Side-Channel for Key Delivery):** Delivers the secret $K = (M, N, a, b, c, d, r_m, r_n)$ and original dimensions (W_0, H_0) separately to the authorized active browser session.

3.2 Encryption & Decryption Lifecycle (Figure 3)

The cryptographic pipeline executes five stages across the server and client:

1. **Padding:** If the source image dimensions (W_0, H_0) are not divisible by (M, N), the server will pad the outer edges until every tile is identical in size.
2. **Key Generation:** The server generates a secret key $K = (a, b, c, d, r_m, r_n)$ satisfying 4 conditions.
3. **Encryption (Forward GRT):** Each Plaintext tiles at $(x, y)_n$ is mapped to its ciphertext coordinate $(x, y)_{n+1}$ via the affine transformation in Eq. (2).
4. **Decryption Precomputation:** Upon receiving secret key K, the client evaluates the 4-stage IGRT corollary (Figure 1) once per axis, storing the inverse mapping in 1D lookup tables in $\mathcal{O}(M + N)$ time.
5. **Decryption (Plaintext Recovery):** The client applies the inverse permutation to restore all tiles to their original coordinates and crops the image back to (W_0, H_0), satisfying $\mathrm{Dec}_K(\mathrm{Enc}_K(P)) = P$.

Note: The complete backend (`grt_server.py`) and frontend (`igrt_client.js`) source code is provided in the Source Code & Artifact Availability repository.

4 Experimental Benchmarking & Analysis

Note: The experimental setup and benchmark scripts used to collect the latency and compression overhead on this Section 4 are hosted in the Source Code & Artifact Availability repository.

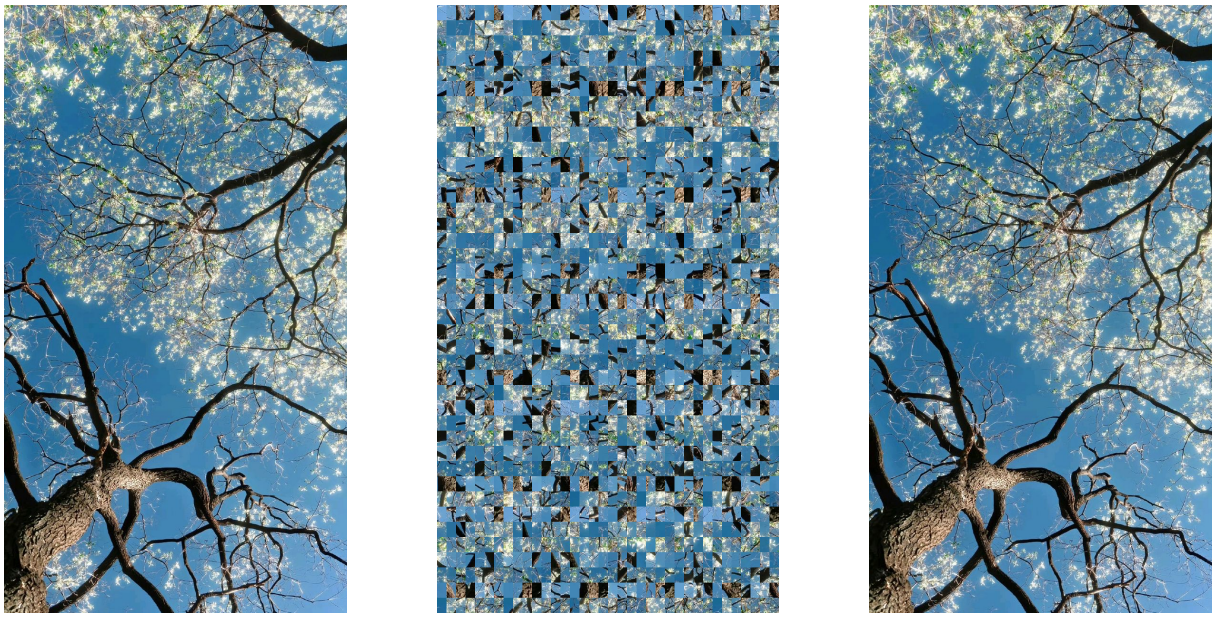
4.1 Decryption Recovery Analysis

A valid symmetric cipher must be strictly invertible: decrypting a ciphertext with the correct key must return the exact original plaintext. We evaluated our implementation across three standard resolutions:

1. **SD 480p (480×854, padded to 480×875):** $M \times N = 30 \times 25$ grid (750 tiles of 16×35 px).

2. **HD 720p** (720×1280)**:** $M \times N = 36 \times 40$ grid (1440 tiles of 20×32 px).
3. **FHD 1080p** (1920×1080)**:** $M \times N = 40 \times 40$ grid (1600 tiles of 48×27 px).

As shown in Figure 4, comparing the original plaintext P and decrypted output $\mathrm{Dec}_K(\mathrm{Enc}_K(P))$ results in zero mismatched pixels and a Mean Squared Error of MSE $= 0.0$, confirming 100% bit-exact recovery.



(a) Original image (720×1280). (b) Scrambled (36×40 grid). (c) Restored (MSE $= 0.0$).

Figure 4: Visual verification of GRT encryption and decryption on HD 720p (36×40 grid, 1440 tiles of 20×32 px).

4.2 Computational Latency & Memory Overhead

Scenario	**Padded Size**	**Grid**	**Tiles**	Enc_K **Time**	Dec_K **Time**	**Total**	**Raw / Peak RAM**	**Ratio**
SD 480p	480×875	30×25	750	3.18 ± 0.14 ms	0.57 ± 0.14 ms	3.74 ms	1230.5/2463.6 KB	2.00×
HD 720p	720×1280	36×40	1440	7.32 ± 0.81 ms	1.11 ± 0.06 ms	8.43 ms	2700.0/5405.4 KB	2.00×
FHD 1080p	1920×1080	40×40	1600	15.13 ± 1.16 ms	2.72 ± 0.28 ms	17.84 ms	6075.0/12162.0 KB	2.00×

Table 1: Encryption/decryption latency (Averaged over 30 independent runs) and memory overhead across SD, HD, and FHD resolutions.

- **Server encryption:** takes < 15.13 ms (~ 100 images/sec per core).
- **Client reconstruction:** takes < 2.72 ms—consuming under 4% of a 16.67 ms, ensuring zero web lag.
- **Constant 2.00× Factor memory:** holds only the plaintext buffer P and ciphertext buffer C in memory at the same time, resulting in an exact 2.00× peak memory ratio for all resolutions.

4.3 Ciphertext Compression Overhead

Scenario (Grid)	Tile Size	PNG ($P \to C$)	WebP ($P \to C$)	JPEG ($P \to C$)
SD 480p (30×25)	16×35 px	904.7 → 918.4 KB (**+1.52%**)	604.0 → 638.2 KB (**+5.65%**)	192.9 → 202.5 KB (**+5.01%**)
HD 720p (36×40)	20×32 px	2047.4 → 2066.9 KB (**+0.95%**)	1353.7 → 1419.2 KB (**+4.83%**)	439.5 → 470.0 KB (**+6.45%**)
FHD 1080p (40×40, Tile GRT)	48×27 px	3497.2 → 3515.6 KB (**+0.52%**)	2291.8 → 2450.4 KB (**+5.65%**)	641.2 → 736.4 KB (**+6.65%**)
FHD 1080p (Pixel Shuffle)	1×1 px	3497.2 → 6070.1 KB (**+73.57%**)	2291.8 → 5116.0 KB (**+123.23%**)	641.2 → 1719.3 KB (**+168.12%**)

Table 2: Ciphertext size expansion comparing tile-level GRT against full pixel-level permutation.

Why Tile Transposition Preserves Compression as We Expected: Shuffling individual pixels will turn an image into random-looking noise that image compressors cannot shrink. By permuting whole tiles (16×35 up to 48×27 px), pixels inside each tile stay together. Only the edges between adjacent tiles change, keeping the expansion between all three resolutions $< 7\%$.

5 Brute-Force Analysis & Limitation

	FHD with 40×40 Grid ($1,600$ tiles)	Cryptographic Implication
Tile Permutation	$1600! \approx 10^{4435}$	Infeasible for brute-force.
Valid Key space	$\lvert\mathcal{K}\rvert \approx 2.46 \times 10^9$ ($\leq 40^6$)	Prevent blind guessing.
Known-Plaintext Attack Resistance	$\mathcal{O}(1)$ solvable with 3 known tiles	GRT is a pure transposition cipher.

Table 3: Brute-force and cryptanalytic evaluation on a 40×40 FHD grid.

- **Limitations:** Because the image must eventually be rendered on the client side, the system cannot prevent OS-level screenshots, headless browser capture, or manual inspection via browser DevTools. Instead, it is designed specifically to prevent direct HTTP bulk scraping and raise the technical cost of automated extraction.

6 Conclusion

By adapting the Generalized Rectangular Transform (GRT) and its 4-stage inverse (IGRT) [3] from pixel-level encryption to tile-level web delivery, our system prevents automated HTTP image scraping while keeping compression overhead low. With the client-side reconstruction under 2.72 ms, show that our system can operate at production speeds in web browsers.

Source Code & Artifact Availability

The full implementation and automated benchmark suite are publicly hosted on our group project GitHub repository:

- **Repository URL:** https://github.com/conca979/cryptography
- **Submission Commit:** `76cde7e`
- **Core Assets:** `grt_server.py` (Python backend),`igrt_client.js` (Client decoder) and "benchmark/" folder (Experimental benchmark setup)

References

[1] E. Uzun, "Scraping Relevant Images from Web Pages without Download," *ACM Transactions on the Web*, vol. 18, no. 1, Art. no. 1, pp. 1–27, Oct. 2023. [Online]. Available: https://doi.org/10.1145/3616849

[2] S. Shan, J. Cryan, E. Wenger, H. Zheng, R. Hanocka, and B. Y. Zhao, "Glaze: Protecting Artists from Style Mimicry by Text-to-Image Models," in *Proc. 32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA, USA, 2023, pp. 2187–2204. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/shan

[3] J. Lan and X. Wang, "Generalized Rectangular Transform and Its Application in Image Encryption," *IEEE Access*, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10909497

[4] V. I. Arnold and A. Avez, *Ergodic Problems of Classical Mechanics*. New York, NY, USA: Benjamin, 1968. [Online]. Available: https://doi.org/10.1016/j.chaos.2003.12.022

[5] R. Salkowitz, "Artists Built A Site To Escape AI. Scrapers Are Coming For It Anyway," *Forbes*, Aug. 23, 2026. [Online]. Available: https://www.forbes.com/sites/robsalkowitz/2026/08/23/artists-built-a-site-to-escape-ai-scrapers-are-coming-for-it-anyway/